최종 발표 대본 (2026-10-26, 모델 선정 추가 버전) · 한국어 AI 합성 음성 탐지 웹서비스

슬라이드 순서: NII_최종발표_20261026_모델선정추가.pptx 기준 (12장)
목표 시간: 약 9분 30초 (10분 이내)
굵은 글씨 = 슬라이드별 핵심 문장

---

1 한국어 AI 합성 음성 탐지 웹서비스

약 10초

안녕하세요, 20223324 고승우입니다. 저는 한국어 음성 파일을 올리면 AI로 합성된 흔적을 찾아 주는 웹서비스를 만들었습니다. **특히 진짜 녹음 속에 끼워 넣은 짧은 AI 음성의 위치를 찾는 데 집중했습니다.**

2 서비스 개발 배경

약 50초

AI 음성은 이미 범죄에 쓰이고 있습니다. LG유플러스는 자사 서비스가 상용화 후 약 한 달 동안 위변조 의심 음성을 약 5,500건 탐지했다고 발표했습니다. 다만 범죄로 확정된 건수는 아닙니다. 익시오나 후후 같은 기존 서비스는 통화 중 실시간 탐지에 초점이 있어서, 이미 받은 음성 메시지나 녹음 파일은 확인하기 어렵습니다. 그래서 저는 받은 파일을 올려 분석하는 서비스를 만들었고, 그중에서도 더 고도화된 공격에 집중했습니다. **진짜 녹음 중간에 '지금 송금해 줘' 같은 한 적 없는 말을 1~2초 끼워 넣는 공격입니다.** 녹음 대부분이 진짜라서 파일 전체로 판정하면 놓치기 쉽고, 녹음 증거 조작에도 쓰일 수 있습니다. 이 예시는 가상 시나리오입니다. 이 공격까지 잡을 수 있는 탐지 서비스를 어떻게 만들었는지 순서대로 설명드리겠습니다.

3 모델 선정: 한국어로 학습시킬 모델 찾기

이런 서비스를 만들려면 먼저 탐지 모델이 필요했습니다. 그래서 공개된 탐지 모델 6개를 한국어 음성 400개로, 원본, 전화 음질, 잡음, 전화와 잡음을 섞은 4가지 조건에서 직접 시험해 봤습니다. 상위 4개는 서로 차이가 거의 없었고, 잡음 조건까지 넣으면 모두 오류율이 약 20% 이상이라 한국어에 그대로 쓰기에는 부족했습니다. **그래서 공개 모델 하나를 한국어 데이터로 추가 학습하기로 했고, 지금 성능보다 학습으로 개선할 수 있는지를 기준으로 골랐습니다.** SSL-AASIST는 원래 성능은 낮은 편이었지만, 학습 코드가 공개돼 있고 LoRA 같은 기법을 바로 적용할 수 있었습니다. 또 원본 음질에서 AI 음성은 가장 적게 놓쳤고 실제 음성을 AI로 잘못 경고하는 문제가 컸는데, 한국어 실제 음성으로 학습하면 고칠 수 있다고 봤습니다. 반면 NII는 당시 실행 파일만 있어서 직접 학습시킬 수 없었습니다. 1인 5주 일정 안에 학습과 평가까지 할 수 있는지도 함께 봤습니다.

4 데이터셋 구성과 처음 보는 생성기 문제

약 35초

SSL-AASIST를 한국어로 추가 학습하려면 한국어 데이터가 필요해서 데이터셋부터 만들었습니다. 같은 대본으로 실제 음성과 AI 음성을 쌍으로 만들고, 화자와 녹음 단위로 TRAIN, DEV, CALIB, FINAL을 나눴습니다. 생성 유형은 일반 TTS, 복제 TTS, 음성 변환 세 가지입니다. 중요한 점은 생성기도 나눴다는 것입니다. 학습에는 Supertonic, XTTS, RVC만 쓰고, DEV에는 처음 보는 Melo, OpenVoice, Seed-VC를 섞고, 최종 시험 FINAL은 전부 처음 보는 Qwen3-TTS, CosyVoice3, kNN-VC로 만들었습니다. **오른쪽은 제가 추가 학습한 SSL-AASIST 모델의 결과인데, 학습한 Supertonic은 0.914였지만 처음 보는 Melo는 0.588, FINAL은 0.623으로 떨어졌습니다.** 처음 봐도 높은 OpenVoice 같은 경우도 있어서 경향으로 봐야 합니다.

5 추가 학습 실험: 무엇이 효과 있었나

약 50초

SSL-AASIST는 추가 학습과 파인튜닝으로 성능을 끌어올려 쓰려고 했습니다. 그래서 후단 학습, LoRA, RawBoost를 단계별로 적용했습니다. 처음 보는 생성기만 있는 FINAL에서 후단 학습은 0.344를 0.610으로 올려 효과가 있었고, LoRA는 0.678까지 더 올렸지만 신뢰구간이 0을 포함해서 확정된 향상은 아닙니다. RawBoost는 이 설정에서는 오히려 0.623으로 떨어졌습니다. 이렇게 성능은 올랐지만, 처음 보는 생성기로 만든 음성에는 여전히 너무 약했습니다. **반면 다른 후보였던 NII는 처음 보는 생성기에도 0.982로 준수한 성능을 유지해서, NII로 바꿔 개발했습니다.** 차이가 난 이유로는 학습 데이터의 규모와 다양성이 그럴듯해 보입니다. 제 가짜 음성 데이터는 3.6시간, NII는 18,280시간입니다. 다만 구조와 학습 방식도 달라서 원인을 하나로 단정할 수는 없습니다. 그렇다고 실험이 쓸모없지는 않았습니다. 무엇이 효과 있고 없는지 확인했고, 이때 만든 학습 경로가 뒤에 나올 재학습 프레임워크의 바탕이 됐습니다. 이 비교는 모델을 고르기 전에 결과를 본 탐색적 비교라서 독립 평가가 더 필요합니다. 이렇게 판정 모델은 NII로 정했는데, NII를 쓰면서 새로운 문제가 보였습니다.

6 삽입 탐지: 경계를 찾고, 구간별로 판정

약 55초

NII는 파일 전체의 특징을 보고 그 파일이 AI 음성인지 사람 음성인지 판단하는 모델입니다. 그래서 배경에서 말씀드린 것처럼 일부 구간만 삽입된 파일에서는 성능이 떨어졌습니다. 4초 창 점수를 평균하는 방식으로는 TRAIN으로 만든 삽입 실험에서 AI 경고가 0%였습니다. 그래서 NII를 어떻게 활용하면 삽입된 음성을 찾아낼 수 있을지 여러 실험을 했습니다. 가장 의심스러운 4초 창만 보는 방법, NII 내부 특징이 갑자기 바뀌는 곳을 찾는 방법 등을 시험했고, 경계로 잘라서 구간마다 NII로 판정하면 효과가 있다는 것을 확인했습니다. **이 과정에서 NII 혼자서는 삽입된 음성을 잘 찾지 못한다는 것을 알게 됐고, 이를 해결하기 위해 음성 삽입과 편집 경계를 찾아내는 BAM이라는 모델을 찾아 함께 쓰기로 했습니다.** 그런데 공개 BAM은 한국어 편집 경계를 절반 정도밖에 못 찾았습니다. 데이터 문제라고 보고 한국어 편집 데이터로 GPU에서 13분 재학습했더니, ±0.3초 안 경계 재현율이 95.8%, 헛경계는 10초당 0.28개가 됐습니다. 오른쪽 그래프에서 고정 4초 창 대비 AUC가 1초 삽입은 0.595에서 0.728로, 2초 삽입은 0.736에서 0.887로 올랐고, 정답 경계로 잘랐을 때의 참고값에 가까워졌습니다. 녹음 묶음은 분리했지만 생성기는 학습과 같은 계열이고, 합성으로 만든 편집이라는 한계가 있습니다. 그래서 서비스를 두 가지 검사로 만들었습니다. 첫째는 음성 전체가 AI로 되어 있는지 판단하는 전체 검사이고, 둘째는 AI가 삽입된 구간이 있는지 확인하는 부분 검사입니다.

7 입력 데이터 처리 파이프라인

약 50초

이 두 검사가 실제로 어떻게 동작하는지 보여 드리겠습니다. 업로드한 음성은 먼저 공통 전처리를 거칩니다. 형식과 크기를 확인하고, PCM으로 디코딩하고, 16kHz 모노로 바꾸고, 앞뒤 무음을 자르고, 음량을 맞춘 뒤, 4초에서 120초 사이인지 확인합니다. NII가 16kHz 파형을 입력으로 받는 모델이라 모든 형식을 같은 조건으로 바꿉니다. 그다음 두 가지 검사를 합니다. 왼쪽 첫째, 전체 검사는 파일을 4초 창으로 나누되 2초씩 겹쳐서 옮겨 가며 창마다 NII 점수를 내고, 그 평균으로 음성 전체가 AI인지 판정합니다. 하지만 주황색처럼 1초짜리 AI 삽입은 4초 창 안에서 평균에 묻히기 쉽습니다. 그래서 오른쪽 둘째, 부분 검사를 함께 돌립니다. 먼저 BAM이 0.16초마다 편집 경계일 확률을 내고, 0.5를 넘는 봉우리를 경계로 봅니다. 그 경계로 음성을 구간 A, B, C로 자르고, 구간 길이 그대로 NII가 판정합니다. **그러면 1초짜리 B 구간만 따로 높은 확률이 나와서 주황색으로 표시됩니다.** 이 그림의 숫자는 이해를 돕는 예시이고, 실제 결과는 시연에서 보여 드리겠습니다.

8 시스템 구성

약 45초

이 두 검사를 실제 서비스로 만든 시스템 구성입니다. 사용자가 올린 음성 파일이 어떤 모듈을 거쳐 가는지 보여 드리겠습니다. 먼저 사용자가 브라우저에서 음성 파일을 고르고 분석하기를 누르면, 파일이 접속 토큰과 함께 API 서버로 갑니다. API 서버인 app.py는 토큰과 파일 크기를 확인하고 파일을 메모리로 받아 분석을 요청합니다. 전처리 모듈이 파일을 PCM으로 바꾸고 16kHz 모노, 앞뒤 무음 제거, 음량 맞춤을 합니다. **그다음 전체 검사는 NII로 파일 전체를, 부분 검사는 BAM으로 경계를 찾은 뒤 구간마다 NII로 판정합니다.** 서버는 한 대이고, BAM만 GPU에서, 나머지는 CPU에서 돕니다. 점수는 미리 TRAIN 데이터로 계산해 둔 보정값으로 확률로 바꿔 결과 JSON으로 돌려주고, 브라우저가 타임라인과 표를 그립니다. 구간 재생은 서버에서 음성을 다시 받지 않고 내 PC의 원본 파일로 재생합니다. 분석이 끝난 음성은 지금은 바로 삭제되고 서버에 저장되지 않습니다. 원래 설계에서는 학습 이용에 동의한 음성만 보관해서, 사람 검토, 데이터셋 버전, 학습, 평가를 거쳐 새 모델로 바꾸는 재학습 경로로 보내도록 했습니다. 이 점선 부분은 코드로 만들어 두었지만 실제 서비스에서는 아직 쓰지 않았습니다.

9 시연: AI 삽입과 사람 삽입 비교

이제 부분 검사를 직접 보여 드리겠습니다. (1) 먼저 '이 데모는 어떻게 판정하나요?'를 잠깐 펼쳐 보여 줍니다(20초). (2) 03_rvc_mid_1s_AI삽입.wav를 올립니다. 사람 말 중간에 1초짜리 음성 변환 음성을 넣은 파일입니다. **기존 방식인 전체 검사는 'AI 음성 증거가 낮음'으로 놓치지만, 부분 검사는 2.64초에서 3.60초 구간을 AI 합성 의심으로 표시합니다.** 이 구간의 모델 추정 확률은 99.5%입니다. 정확도가 99.5%라는 뜻은 아니고, TRAIN 합성 데이터로 보정한 추정값입니다. 재생 버튼을 눌러 그 구간을 들려 드립니다(50초). (3) 같은 위치에 다른 사람의 실제 목소리를 넣은 03_rvc_mid_1s_사람삽입.wav를 올립니다. 경계는 찾지만 AI로 표시하지 않습니다. 최고 2%입니다. 경계를 찾는 것과 AI인지 판정하는 것은 서로 다른 일이라는 점을 보여 줍니다(40초).

[시연 2분 10초]

[대비책: 접속이 안 되면 이 슬라이드의 캡처와 녹화 영상으로 진행]

[시연 전: start_demo.ps1을 발표장 네트워크에서 실행, 1~2분 뒤 접속]

10 남은 문제

약 45초

남은 문제는 네 가지입니다. 첫째, 오표시입니다. 학습에 쓰지 않은 화자 10명으로 간이 시험을 했더니, 편집하지 않은 실제 음성 10개 중 1개에 의심 표시가 나왔고, 삽입 파일 60개에서 AI가 아닌 구간을 19번 잘못 표시했습니다. 증거를 검토하는 용도에서는 직접 문제가 되는 부분입니다. 둘째, 처음 보는 음성 변환입니다. Windows 기본 TTS는 1초 삽입 10개를 모두 잡았지만, 최신 음성 변환인 Seed-VC 1초 삽입은 10개 중 4개만 표시했습니다. **경계는 60개 모두 찾았으므로 약점은 경계 찾기가 아니라 구간을 판정하는 NII 쪽입니다.** 셋째, 전처리입니다. 무손실 형식이어도 16kHz 변환, 모노 변환, 음량 조정, 무음 제거로 모델 입력은 바뀝니다. 실제로 WAV를 MP3로 바꾸자 판정이 바뀐 사례가 있었는데, 원인이 무엇인지는 아직 확인하지 못했습니다. 넷째, AI로 데이터를 만들기는 쉽지만 정제하기는 어렵기 때문에, 기부 데이터로 학습을 오염시키는 공격이 가능합니다.

11 개선사항

약 45초

개선사항은 두 가지입니다. 첫째, 재학습 프레임워크입니다. 사용자 동의, 사람 검토, 데이터셋 버전, 학습, 평가 게이트, 승격과 롤백 순서로 구상하고 코드로 만들어 두었지만, 실제 서비스에서 사용해 보지는 못했습니다. 학습 실행 단계는 지금 꺼 두었습니다. 앞에서 말한 데이터 오염에 대비해, 라벨은 사람만 붙이고, 실제 음성을 잘못 경고하는 비율이 늘면 새 모델로 바꾸지 않도록 설계했습니다. 새 VC가 나오면 이 경로로 데이터를 모아 BAM 때처럼 파인튜닝이나 LoRA로 보강할 수 있습니다. 둘째, 모델 경량화와 비용입니다. 지금 부분 검사는 GPU 서버에서 돌아가서 시간당 약 1.24달러, 한 달 내내 켜 두면 약 905달러가 듭니다. BAM을 CPU에서 돌리면 120초 파일에 약 2분, 메모리 8GB 이상이 필요해서 지금 공개 서버에는 못 올립니다. **작은 모델로 증류하거나 가벼운 경계 판정부를 다시 학습해서 공개 서버와 같은 CPU 서버로 옮기면 시간당 약 0.21달러, 한 달 약 153달러로 약 83% 줄일 수 있을 것으로 예상합니다.** 다만 처음 시도한 가벼운 후보 BDR은 이번 학습 설정에서 경계를 찾지 못해서, 경량화는 앞으로의 과제입니다.

12 기대효과

약 25초

기대효과는 세 가지입니다. 받은 파일 속 짧은 조작 구간을 위치와 확률로 짚어 주는 판단 근거가 되고, 음성을 저장하지 않아 부담 없이 쓸 수 있으며, 새로운 공격이 나와도 오늘 보여 드린 것처럼 분석하고 보강할 수 있는 구조를 갖췄습니다. 판정을 대신 내려 주는 도구가 아니라 판단을 돕는 근거라는 점도 덧붙입니다. 마지막으로, AI가 빠르게 발전하는 만큼 고도화된 공격도 계속 새로 나오고 있습니다. **이에 대응하려면 계속 성장시킬 수 있는 탐지 서비스가 필요하다고 생각합니다.** 들어 주셔서 감사합니다.

[예상 질문: 정확도는? → 통제된 합성 실험 결과이고 독립 평가(FINAL v3)가 더 필요 / 왜 NII? → 3·5번 슬라이드 / 왜 음성을 저장하지 않나? → 기간과 개인정보 위험, 학습 동의를 받지 않음]